

6.1 이미지 분류를 위한 신경망

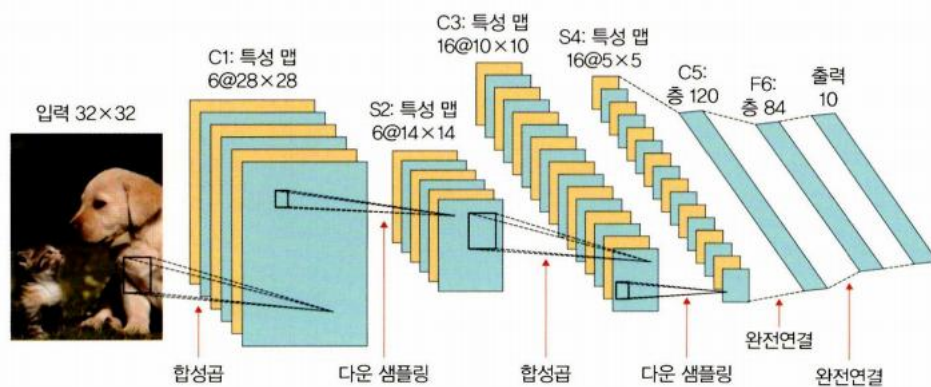
이미지 분류(classification) : 특정 대상이 영상 내에 존재하는지 여부를 판단

6.1.1 LeNet-5

LeNet-5

- 얀 르쿤(Yann LeCun)이 개발한 합성곱 신경망
- 이미지 분류를 위한 초기 CNN 구조 중 하나
- 합성곱층 + 풀링층 반복 → 완전연결층 → 분류 결과 출력

▼ 그림 6-1 LeNet-5



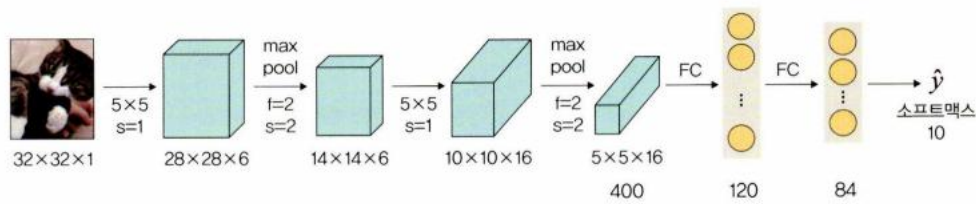
LeNet-5 구조

▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

-
- 소프트맥스
- 합성곱/풀링을 거치며 이미지 크기 ↓, 채널 수 ↑

▼ 그림 6-2 LeNet-5 예제 신경망



이미지 전처리

훈련 데이터와 검증 데이터에 서로 다른 전처리 적용

transforms.Compose

- 여러 이미지 변환 방법을 순서대로 묶어서 적용

RandomResizedCrop

- 이미지의 일부를 임의로 자른 뒤 원하는 크기로 변환
- `scale=(0.5, 1.0)` 사용

RandomHorizontalFlip

- 이미지를 좌우로 뒤집음
- 원본과 다른 이미지 생성 → 훈련 데이터 다양성 증가

ToTensor

- PIL 이미지 / NumPy 배열 → Tensor
- 이미지 채널 순서도 Tensor 형태에 맞게 변경

Normalize

- 평균(mean), 표준편차(std)를 이용해 이미지 정규화
- ImageNet에서 사용하는 값 적용

__call__()

- 클래스 객체를 함수처럼 호출할 수 있게 함
- phase 값에 따라 train / val 전처리 적용

이미지 데이터셋 분리

Cat / Dog 이미지 경로를 가져온 뒤 데이터 분리

- 훈련 데이터 : 400개
- 검증 데이터 : 92개
- 테스트 데이터 : 10개

sorted()

- 데이터를 정렬된 리스트로 반환

os.path.join()

- 여러 경로를 하나의 경로로 결합

os.listdir()

- 지정한 디렉터리의 파일 목록 반환

cv2.imread()

- 이미지 파일 읽기
- 읽을 수 없는 이미지 → None
- None이 아닌 이미지만 사용

random.seed()

- 난수 생성 기준값 설정
- 같은 seed → 같은 난수 결과
- 데이터 셔플 결과를 동일하게 유지

테스트 이미지 출력

cv2.cvtColor

- OpenCV 기본 색상 순서 : **BGR**
- 일반 이미지 출력에 사용하는 **RGB**로 변환

os.path.normpath

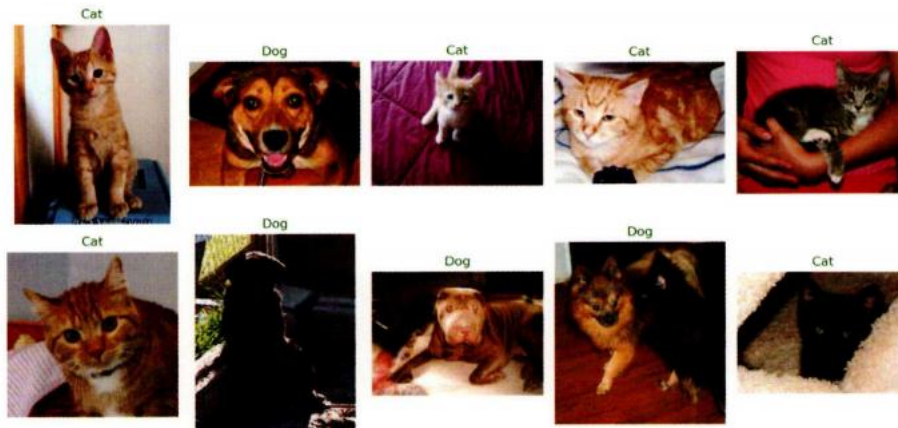
- 파일 경로를 운영체제 형식에 맞게 정규화

- 이미지 경로에서 실제 레이블 cat, dog 추출에 사용

실제 레이블 / 예측 레이블

- 예측값 존재 → predicted_labels 사용
- 예측값 없음 → 실제 레이블 사용

▼ 그림 6-4 테스트 데이터셋의 이미지



Dataset 클래스

torch.utils.data.Dataset 상속 → 사용자 정의 데이터셋 생성

`__init__()`

- 이미지 파일 목록 저장
- 전처리 방법 저장
- phase 저장

`__len__()`

- 전체 데이터 개수 반환

`__getitem__()`

- 특정 index의 데이터 가져오기
- 이미지 읽기
- 이미지 전처리
- 레이블과 함께 반환

레이블

- dog $\rightarrow$ 1
 - cat $\rightarrow$ 0
-

전처리 변수

- 이미지 크기 : 224
- 평균 : (0.485, 0.456, 0.406)
- 표준편차 : (0.229, 0.224, 0.225)
- batch size : 32

전처리 후 이미지 형태

DataLoader

DataLoader : Dataset의 데이터를 배치 단위로 불러옴

dataset

- 사용할 데이터셋

batch_size

- 한 번에 가져올 데이터 개수
- 교재 : 32

shuffle

- 데이터를 섞어서 불러올지 설정
 - 훈련 : shuffle=True
 - 검증 : shuffle=False
-

LeNet 네트워크

구조

$Conv2d \rightarrow ReLU \rightarrow MaxPool2d \rightarrow Conv2d \rightarrow ReLU \rightarrow MaxPool2d \rightarrow FC \rightarrow FC \rightarrow Softmax$

Conv2d

첫 번째 합성곱층

- 입력 채널 : 3
- 출력 채널 : 16
- 커널 : 5×5

두 번째 합성곱층

- 입력 채널 : 16
- 출력 채널 : 32
- 커널 : 5×5

Conv2d 계층에서의 출력 크기 구하는 공식

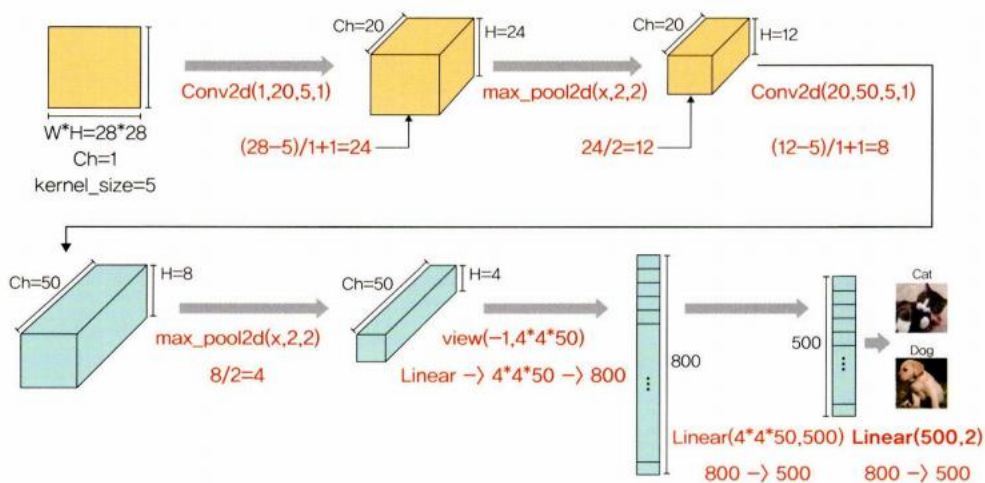
$$\text{출력 크기} = (W - F + 2P) / S + 1$$

- W : 입력 데이터의 크기(input_volume_size)
- F : 커널 크기(kernel_size)
- P : 패딩 크기(padding_size)
- S : 스트라이드(strides)

MaxPool2d 계층에서의 출력 크기 구하는 공식

$$\text{출력 크기} = IF / F$$

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



완전연결층

- 합성곱/풀링을 거친 feature map → 1차원으로 변경
 - out.view(out.size(0), -1) 사용
-

torchsummary

torchsummary

- 모델의 네트워크 구조 확인
- 각 계층의 출력 형태 확인
- 파라미터 수 확인

summary(model, input_size=(3,224,224))

- 첫 번째 파라미터 : 모델
 - 두 번째 파라미터 : 입력 크기
-

학습 가능한 파라미터

trainable parameters

- requires_grad=True인 파라미터
 - 학습 과정에서 업데이트되는 값
-

옵티마이저 / 손실 함수

SGD

- 모델 파라미터 업데이트에 사용
- learning rate = 0.001
- momentum = 0.9

CrossEntropyLoss

- 분류 문제의 손실 함수
- Cat / Dog 두 클래스 분류에 사용

모델 학습

훈련 과정 : train / val 반복

model.train()

- 모델 → 훈련 모드

model.eval()

- 모델 → 평가 모드

optimizer.zero_grad()

- 이전 gradient 초기화

torch.set_grad_enabled()

- 훈련 단계에서 gradient 계산

loss.backward()

- 손실을 이용해 역전파

optimizer.step()

- 계산한 gradient로 파라미터 업데이트

epoch loss

- 전체 데이터에 대한 평균 손실

epoch accuracy

- 전체 데이터 중 올바르게 예측한 비율

reduction

reduction : 여러 데이터의 손실값을 어떻게 반환할지 결정
교재

reduction='mean'

→ 전체 손실의 평균값 사용

테스트 데이터 예측

torch.no_grad()

- gradient 계산 X
- 테스트에서 가중치 업데이트 필요 X

unsqueeze(0)

- Tensor에 새로운 차원 추가
- 이미지 한 장에 배치 차원 추가

F.softmax

- 출력값 → 클래스별 확률
- 두 번째 클래스의 확률 사용

예측 결과

- DataFrame으로 저장
- CSV 파일로 저장
- 0.5 기준 Cat / Dog 분류

▼ 그림 6-7 테스트 데이터셋에 대한 LeNet 모델의 예측 결과

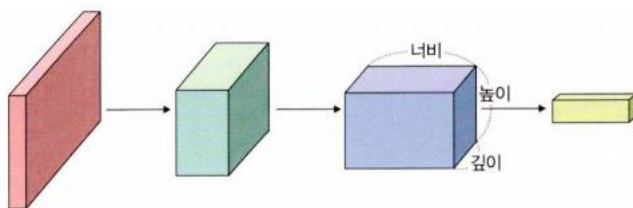


6.1.2 AlexNet

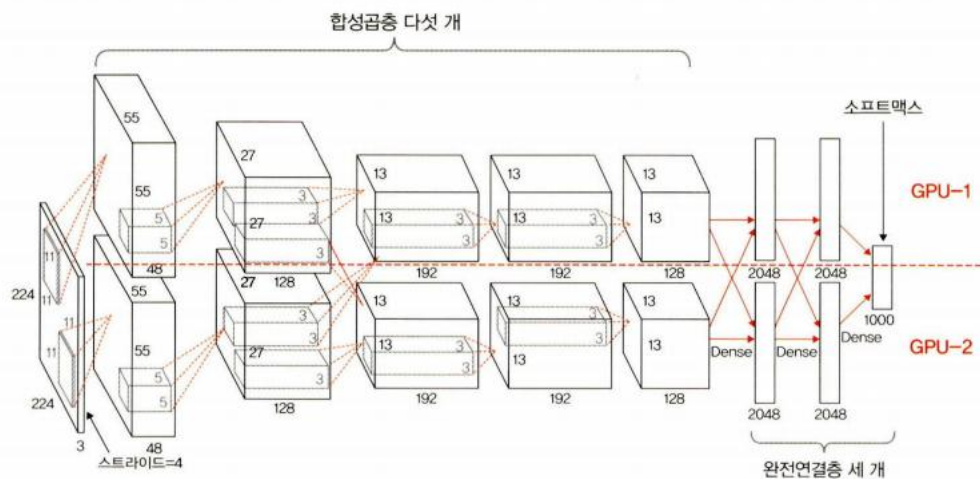
AlexNet

- ImageNet 기반 영상 인식 대회 **ILSVRC 2012 우승**
- LeNet-5보다 깊은 CNN 구조
- 이미지 크기 ↓ / 채널 수 ↑
- 합성곱층 + 최대 풀링층 + 완전연결층 사용
- 활성화 함수 : ReLU
- GPU 두 개를 이용하도록 구성

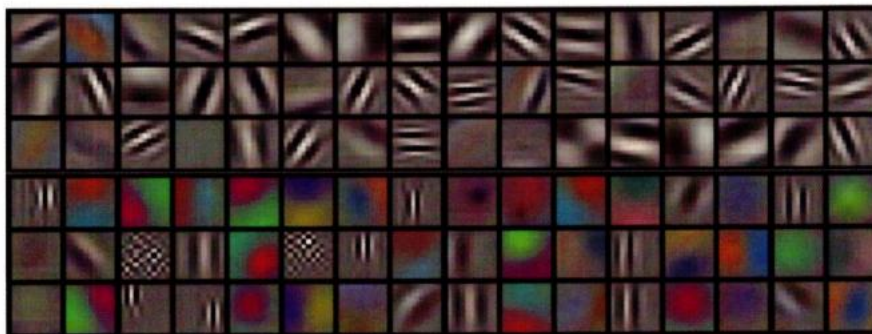
▼ 그림 6-8 CNN 구조



▼ 그림 6-9 AlexNet 구조



▼ 그림 6-10 AlexNet GPU-1 · 2 적용 결과



AlexNet 입력

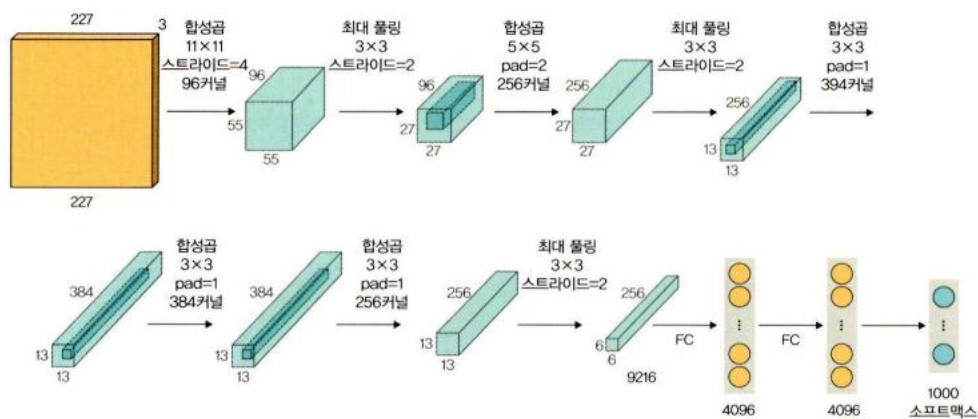
입력 이미지 : 227X227X3

- RGB 이미지
- 네트워크 변수 약 6,600만 개

주요 계층

- 합성곱층
- ReLU
- 최대 풀링층
- 완전연결층

▼ 그림 6-11 AlexNet 예제 네트워크



inplace

`nn.ReLU(inplace=True)`

- 연산 결과를 새로운 공간에 저장 X
- 기존 데이터를 직접 변경

AdaptiveAvgPool2d

`AvgPool2d`

- 커널 크기 / stride 등을 직접 지정

`AdaptiveAvgPool2d`

- 원하는 출력 크기를 지정

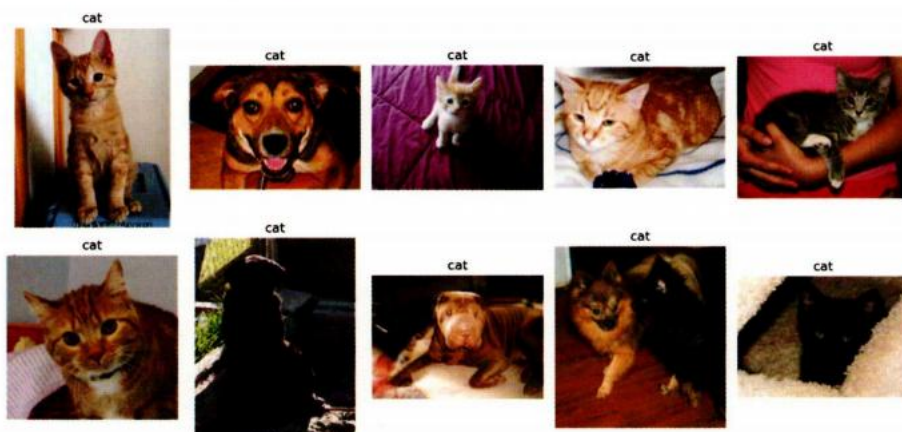
AlexNet

- 출력 크기 (6,6)으로 설정

AlexNet 학습 과정

1. 데이터 전처리
2. Dataset 생성
3. DataLoader 생성
4. 모델 정의
5. 옵티마이저 / 손실 함수 정의
6. 훈련 / 검증
7. 테스트
 - optimizer : SGD
 - loss : CrossEntropyLoss
 - LeNet보다 구조가 복잡하고 파라미터 수가 많음
 - 학습 시간 증가

▼ 그림 6-12 테스트 데이터셋에 대한 AlexNet 모델의 예측 결과

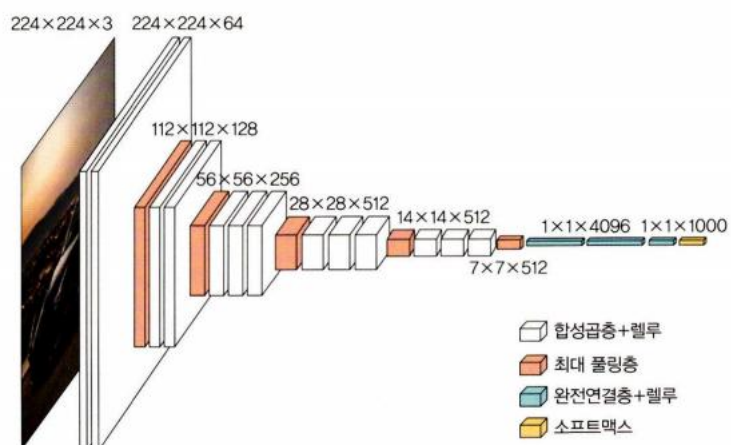


6.1.3 VGGNet

VGGNet

- 카렌 시모니안(Karen Simonyan), 앤드류 지서먼(Andrew Zisserman)이 제안
- 네트워크의 **깊이**가 모델 성능에 미치는 영향 확인
- 종류
 - VGG11
 - VGG13
 - VGG16
 - VGG19
- 주로 3×3 합성곱 커널 사용
- 최대 풀링 : 2×2

♥ 그림 6-13 VGG16 구조



객체 복사

단순 대입

```
copy_o = original
```

- copy_o, original → 같은 객체 참조
- 한쪽 값 변경 → 다른 쪽에도 영향

얕은 복사(shallow copy)

```
copy.copy()
```

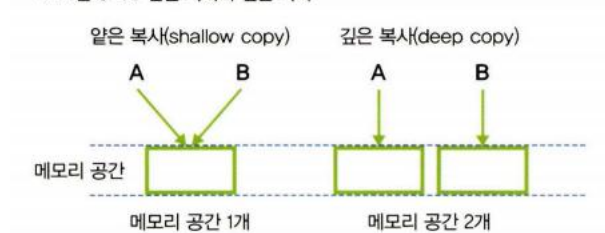
- 새로운 객체 생성
- 내부 객체까지 모두 새롭게 복사하는 것은 아님

깊은 복사(deep copy)

`copy.deepcopy()`

- 내부 객체까지 새로 복사
- 원본 / 복사본 독립적으로 사용 가능

▼ 그림 6-14 얕은 복사와 깊은 복사



VGG 모델 구성

features

- 합성곱층 / 풀링층
- 이미지 특징 추출

avgpool

- feature map 크기 조정

classifier

- 완전연결층
- 최종 분류 수행
- ReLU / Dropout 사용

VGG 계층 구성

VGG11 / VGG13 / VGG16 / VGG19

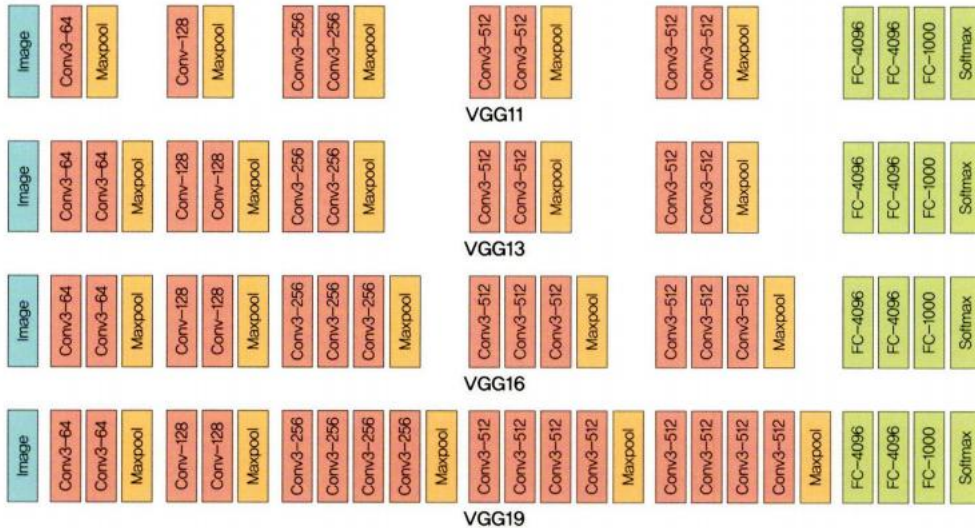
- 합성곱층 개수가 서로 다름

- 리스트 형태로 네트워크 구조 정의

리스트

- 숫자 → 합성곱층의 출력 채널 수
- 'M' → MaxPool2d

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



get_vgg_layers()

config 리스트 → 실제 VGG 계층 생성

- 값 = 'M' → MaxPool2d
- 값 = 숫자 → Conv2d
- batch_norm=True
 - Conv2d
 - BatchNorm2d
 - ReLU

assert

- 조건이 참인지 확인
- 조건 = False → 오류 발생

- VGG config 값이 'M' 또는 정수인지 확인
-

isinstance

- 객체의 자료형 확인

isinstance(1, int)

→ True

VGG

- config 값이 정수인지 확인
-

배치 정규화(Batch Normalization)

BatchNorm2d

- 각 계층의 입력 데이터를 일정한 분포로 맞춤
 - 교재 : batch_norm=True
-

사전 훈련된 VGG11

models.vgg11_bn(pretrained=True)

- torchvision에서 제공하는 VGG11 사용
 - pretrained=True
→ 이미 학습된 가중치 사용
-

데이터 전처리

훈련 데이터

- Resize
- RandomRotation
- RandomHorizontalFlip

- ToTensor
- Normalize

테스트 데이터

- Resize
- ToTensor
- Normalize

RandomRotation

- 이미지를 일정 범위 내에서 회전
- 훈련 데이터 다양성 증가

ImageFolder

torchvision.datasets.ImageFolder

- 클래스별 폴더 구조의 데이터셋 사용
- 폴더 이름 기준으로 클래스 자동 구분

▼ 그림 6-16 './chap06/data/catanddog/train' 위치의 하위 폴더

이름	유형
Cat	파일 폴더
Dog	파일 폴더

훈련 / 검증 데이터 분리

random_split()

- 전체 훈련 데이터 일부 → 검증 데이터로 분리

검증 데이터

- 테스트용 transform 적용

정확도 계산

argmax

- 가장 큰 값을 가진 클래스의 **인덱스** 반환

eq

- 두 Tensor 비교
- 같음 → True
- 다름 → False

view_as

- Tensor 크기를 다른 Tensor와 동일하게 변경
-

학습 함수

훈련 순서

1. 입력 / 레이블 → device
2. gradient 초기화
3. 모델 출력 계산
4. loss 계산
5. accuracy 계산
6. 역전파
7. optimizer로 파라미터 업데이트

검증 / 테스트

- torch.no_grad() 사용
 - gradient 계산 X
-

학습 시간

time.monotonic()

- 시작 시간 / 종료 시간 측정
- epoch별 학습 시간 계산

테스트 데이터 예측

저장 내용

- 이미지
- 실제 레이블
- 예측 레이블
- 예측 확률

torch.cat

- 여러 Tensor 연결
- 지정한 차원을 기준으로 합침
- 여러 배치의 이미지 / 레이블 / 확률을 하나로 연결

▼ 그림 6-17 차원(dim)

	Col1	Col2	Col3	
Row1				→ axis=1(dim=1)
Row2				
Row3				

axis=0(dim=0)

max / argmax

max

- 가장 큰 값 반환

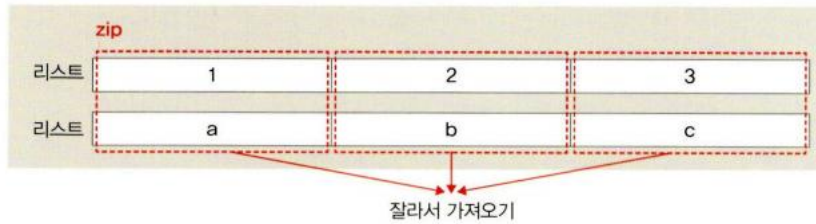
argmax

- 가장 큰 값의 인덱스 반환

zip()

- 여러 리스트에서 같은 위치의 값끼리 묶음
- 이미지 / 실제 레이블 / 예측 확률을 함께 가져올 때 사용

♥ 그림 6-18 zip()의 원리



sort()

- 데이터 정렬
- reverse=True → 내림차순

교재

- 모델의 예측 결과 정렬에 사용

람다 함수

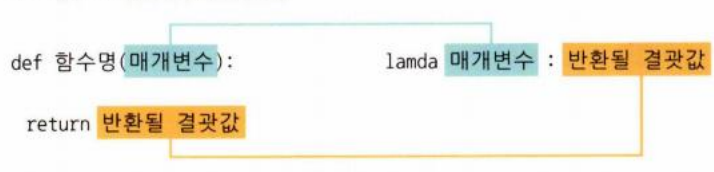
lambda 함수

- 이름 없는 간단한 함수
- 한 줄로 표현

lambda 입력값 : 반환값

- sort()의 key에 사용

♥ 그림 6-19 일반 함수와 람다 함수



이미지 정규화 복원

Normalize 적용 이미지

→ 그대로 출력하면 원래 이미지와 다르게 보임

따라서

- 최소값
- 최대값

을 이용해 이미지 값의 범위 재조정

♥ 그림 6-20 본래 이미지 색상이 왜곡됨



♥ 그림 6-21 본래 이미지



torch.add()

- Tensor 덧셈에 사용
- 메모리 사용 방식 확인

permute

- Tensor 차원 순서 변경
- PyTorch 이미지 Tensor → matplotlib에서 출력 가능한 순서로 변경

예측 결과 시각화

테스트 이미지 중

- 올바르게 분류한 이미지
- 잘못 분류한 이미지

출력

각 이미지

- 실제 레이블

- 예측 레이블

표시

▼ 그림 6-22 테스트 데이터셋에 대한 VGG 모델의 예측 결과



6.1.4 GoogLeNet

GoogLeNet

- 구글에서 개발
- 인셉션(Inception) 모듈 사용
- 하나의 입력에 여러 연산을 병렬로 적용
- 각 결과를 결합하여 다음 계층으로 전달

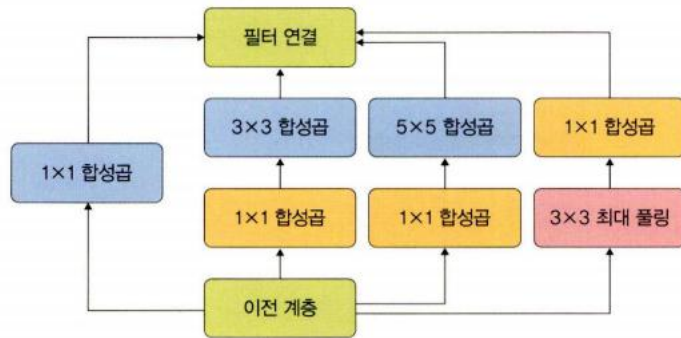
기존 CNN

- 합성곱층 / 풀링층 → 순차적 연결

GoogLeNet

- 서로 다른 크기의 합성곱 / 풀링 → 병렬 수행

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



인셉션 모듈

동시에 수행하는 연산

- 1×1 합성곱
- 1×1 합성곱 → 3×3 합성곱
- 1×1 합성곱 → 5×5 합성곱
- 3×3 최대 풀링 → 1×1 합성곱

→ 서로 다른 크기의 필터 사용

→ 다양한 특징 추출

1×1 합성곱

- 연산량 감소에 사용

핵심

- 단순히 계층을 깊게 쌓는 구조 X
- 한 계층에서 여러 연산을 **병렬 수행**
- 결과를 결합해 다음 계층으로 전달